

Docker 3

Part 1 covered the concepts and Part 2 the image and the Compose file. This part is the command line: what each command does, the flags worth remembering, and how to read what comes back.

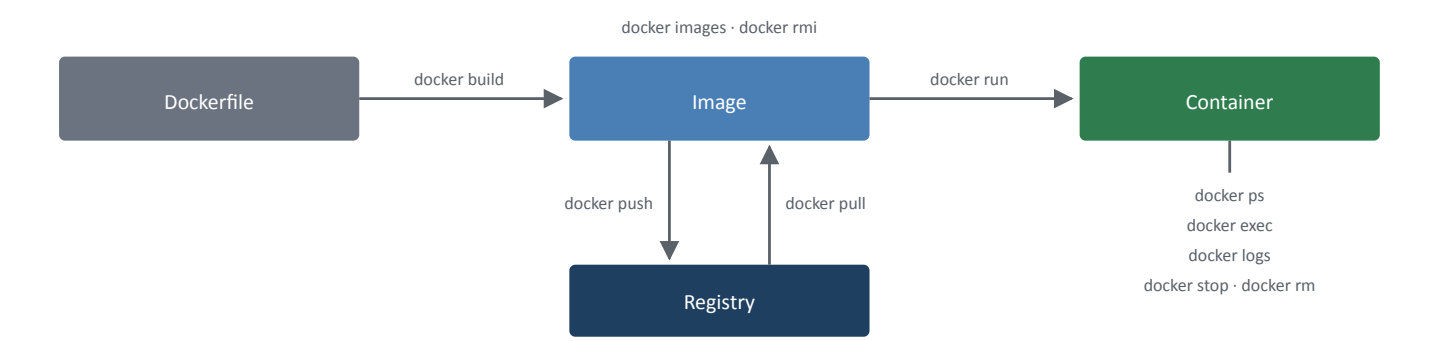
1. The shape of a Docker command

Every command names an object and an action.

```
docker <object> <action> [options] [target]
docker container ls
docker image rm payroll:1.3
```

The short forms came first and are what people actually type. Both work, and the documentation uses the long ones.

Short	Long	Acts on
<code>docker ps</code>	<code>docker container ls</code>	Containers
<code>docker rm</code>	<code>docker container rm</code>	Containers
<code>docker images</code>	<code>docker image ls</code>	Images
<code>docker rmi</code>	<code>docker image rm</code>	Images
<code>docker build</code>	<code>docker image build</code>	Images
<code>docker run</code>	<code>docker container run</code>	Both



Most of the command set is one of these arrows. The rest inspects what is already there, or cleans it up.

`docker <command> --help` is the fastest reference there is, and it is offline.

2. docker build

Builds an image from a Dockerfile and a context.

```
docker build -t payroll:1.4 .
```

Flag	What it does
<code>-t name:tag</code>	Names the result. Repeat it for several tags
<code>-f path</code>	Use a Dockerfile that is not <code>./Dockerfile</code>
<code>--build-arg K=V</code>	Sets an <code>ARG</code> for this build
<code>--no-cache</code>	Rebuilds every layer, ignoring the cache
<code>--target stage</code>	Stops at a named stage in a multi-stage file
<code>--platform</code>	Builds for another architecture, such as <code>linux/amd64</code>
<code>--progress=plain</code>	Prints full build output instead of collapsing it

`--target` is the one that surprises people. It stops the build at a named stage, which is how a test stage runs in a pipeline without shipping.

```
docker build --target test -t payroll:test .
docker build --target runtime -t payroll:1.4 .
```

`--platform` matters on an Apple laptop. A build on an M-series Mac produces `arm64`, and an `amd64` server cannot run it.

```
docker build --platform linux/amd64 -t payroll:1.4 .
```

3. docker run

Creates a container from an image and starts it. It is `pull` if missing, then `create`, then `start`.

```
docker run -d --name payroll -p 8080:8080 --restart unless-stopped payroll:1.4
```

Flag	What it does
<code>-d</code>	Detached. Runs in the background
<code>-it</code>	Interactive with a terminal. For a shell
<code>--rm</code>	Deletes the container when it stops

Flag	What it does
<code>--name</code>	A fixed name instead of a random one
<code>-p host:container</code>	Publishes a port to the host
<code>-e K=V</code>	One environment variable
<code>-v name:/path</code>	Mounts a volume or a host directory

The rest come up less often, and each answers one specific need:

Flag	What it does
<code>--env-file .env</code>	A file of variables instead of many <code>-e</code> flags
<code>--network name</code>	Joins a user-defined network
<code>-u 1000:1000</code>	Runs as a different user
<code>-w /app</code>	Sets the working directory
<code>--memory 512m</code>	A hard memory limit
<code>--cpus 1.5</code>	A CPU limit
<code>--entrypoint sh</code>	Replaces the image's entrypoint

Four of these carry almost every real command, and each is worth saying out loud on its own.

`docker run -d` detaches. The container runs in the background and the terminal comes back with the container id. Without it the terminal is held by the container's output until it stops.

```
docker run -d payroll:1.4
```

`docker run -p` publishes a port. Host port first, container port second. A container with no `-p` is reachable from other containers and not from the laptop.

```
docker run -d -p 8080:8080 payroll:1.4    # localhost:8080 reaches it
docker run -d -p 9090:8080 payroll:1.4    # localhost:9090 reaches the same 8080
```

`docker run -e` sets an environment variable. This is how one image serves every environment, because the configuration arrives at start rather than being built in.

```
docker run -d -e SPRING_PROFILES_ACTIVE=prod \
    -e SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/payroll \
    payroll:1.4
```

docker run -v mounts storage that outlives the container. A named volume on the left, the path inside the container on the right.

```
docker run -d -v pgdata:/var/lib/postgresql/data postgres:16 # named volume
docker run -d -v $(pwd)/config:/app/config:ro payroll:1.4 # host directory
```

Put together, three shapes cover most use:

```
# a throwaway shell in an image, gone when it exits
docker run --rm -it eclipse-temurin:21-jre-alpine sh

# one command, then out
docker run --rm postgres:16 psql --version

# a long-running service, with all four
docker run -d --name payroll -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=prod \
  -v payroll-logs:/app/logs \
  payroll:1.4
```

--rm with **-it** is the habit worth building. Without it, every experiment leaves a stopped container behind, and **docker ps -a** fills with them.

Anything after the image name replaces the command, not the entrypoint.

```
docker run --rm payroll:1.4 --debug # arguments to java -jar app.jar
docker run --rm --entrypoint sh payroll:1.4 # a shell instead
```

4. docker ps

Lists containers. Running ones only, unless asked otherwise.

```
docker ps -a
```

Output:

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS
NAMES				
a3f1c82b9e04	payroll:1.4	"java -jar app.jar"	Up 2 minutes (healthy)	
0.0.0.0:8080->8080/tcp	payroll			
7d20e4b6c115	postgres:16	"docker-entrypoint..."	Exited (137) 4 hours ago	
db				

Three columns carry the information. **STATUS** reads `Up` with the uptime or `Exited` with the code. **PORTS** is empty when nothing is published to the host. **NAMES** is what every other command should refer to, and **COMMAND** is the truncated entrypoint, which is the quickest way to notice that the wrong image is running.

Flag	What it does
<code>-a</code>	All containers, including stopped ones
<code>-q</code>	Ids only. For feeding into another command
<code>-n 5</code>	The last five, whatever their state
<code>-s</code>	Adds the writable layer size
<code>--filter</code>	<code>status=exited</code> , <code>name=payroll</code> , <code>ancestor=payroll:1.4</code>
<code>--format</code>	A Go template, for a narrower table

```
docker ps --filter status=exited -q | xargs docker rm
docker ps --format '{{.Names}}\t{{.Status}}'
```

`(healthy)` in **STATUS** comes from a `HEALTHCHECK`. No healthcheck means no word there, and `Up` then only proves the process has not exited.

5. docker start, stop and restart

These move a container between states without creating a new one.

Command	What it does
<code>docker start name</code>	Starts a created or stopped container again
<code>docker stop name</code>	<code>SIGTERM</code> , then <code>SIGKILL</code> after ten seconds
<code>docker restart name</code>	A stop followed by a start
<code>docker kill name</code>	<code>SIGKILL</code> at once, no shutdown
<code>docker pause name</code>	Freezes every process, keeps memory
<code>docker unpause name</code>	Resumes it
<code>docker rm name</code>	Deletes a stopped container

```
docker stop -t 30 payroll      # wait 30 seconds before the kill
docker rm -f payroll          # kill and delete in one step
```

The ten seconds are the application's shutdown window. A Spring Boot service uses them to finish in-flight requests and close the connection pool, so `-t 30` on a slow service is a real fix rather than a workaround.

docker start keeps everything. The same writable layer, name, ports and volumes come back, because nothing was deleted. That is the difference from **docker run**, which builds a new container every time.

```
docker stop $(docker ps -q)      # stop everything running
docker restart payroll           # the usual first thing to try
```

6. docker exec

Runs a new process inside a container that is already running.

docker exec -it is the form to remember. The **-i** keeps standard input open and the **-t** gives a terminal, and a shell needs both. Without them the shell starts, finds nothing to read and exits at once.

```
docker exec -it payroll sh
```

Flag	What it does
-it	Interactive with a terminal. Needed for a shell
-u root	Runs as another user, useful when the image drops privileges
-w /app	Starts in a different directory
-e K=V	Adds a variable for this process only

The four questions it answers, in order of how often they come up:

```
docker exec payroll env          # what configuration arrived
docker exec payroll ls -la /app  # what files are really there
docker exec payroll ps -ef       # what is running inside
docker exec -it payroll sh       # everything else
```

Use **sh**, not **bash**. Alpine images have no bash, and the error message (`exec: "bash": executable file not found`) reads like something worse than a missing shell.

exec changes nothing permanent. The process runs in the container's writable layer, so a package installed this way disappears on the next **docker run**. Fix the Dockerfile instead.

exec is not **attach**. **attach** connects to the main process, so pressing **Ctrl+C** there stops the container. **exec** starts a separate process and leaving it changes nothing.

7. docker logs

Prints what the container wrote to standard output and standard error.

`docker logs -f` follows the output instead of printing what exists and returning, which is what makes it useful while reproducing a problem. Pair it with `--tail` so a week-old container does not print a week of lines first.

```
docker logs -f payroll          # follow from now
docker logs -f --tail 100 payroll # the last 100 lines, then follow
```

Flag	What it does
<code>-f</code>	Follows new output
<code>--tail N</code>	The last N lines only
<code>--since 10m</code>	Anything newer than that, also <code>--since 2026-09-15T10:00</code>
<code>--until</code>	The other end of a window
<code>-t</code>	Adds timestamps

Logs only work if the application writes to the console. An application writing to `/var/log/app.log` inside the container produces an empty `docker logs` and a file nobody will read. In Spring Boot that means leaving the default console appender alone.

The logs survive a stop and die with a `docker rm`. They sit on the host as a JSON file until then, which is why `--tail` matters on a container that has been up for a week.

```
docker logs payroll 2>&1 | grep -i error
```

8. Image commands

Command	What it does
<code>docker images</code>	Lists local images
<code>docker pull name:tag</code>	Downloads an image
<code>docker push name:tag</code>	Uploads it to a registry
<code>docker tag src target</code>	Adds another name to the same image
<code>docker rmi name:tag</code>	Deletes an image
<code>docker history name:tag</code>	The layers and what made them

```
docker images
```

Output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
payroll	1.4	7c9e2a1f5b3d	12 seconds ago	214MB
payroll	<none>	2b8d41c7e9a0	3 days ago	214MB

A `<none>` tag is a **dangling image**. It is the old image left behind when a tag moved to a rebuild. They accumulate quietly and `docker image prune` removes them.

`docker rmi` fails while a container uses the image, even a stopped one. Remove the container first, or accept that the error is telling the truth.

`docker save -o f.tar name` and `docker load -i f.tar` move an image without a registry, which is how one reaches a machine with no network.

9. docker inspect

Prints everything the daemon knows about an object, as JSON. On its own it is too much, so it is almost always used with `--format`.

```
docker inspect --format '{{.State.Status}}' payroll
docker inspect --format '{{.State.Health.Status}}' payroll
docker inspect --format '{{.RestartCount}}' payroll
docker inspect --format '{{.NetworkSettings.Networks.payroll_default.IPAddress}}'
payroll
docker inspect --format '{{range .Mounts}}{{.Source}} -> {{.Destination}}{{"\n"}}
{{end}}' db
```

Question	Field
Is it healthy?	<code>.State.Health.Status</code>
Why did it exit?	<code>.State.ExitCode</code> , <code>.State.OOMKilled</code>
Has it been crash-looping?	<code>.RestartCount</code>
What is mounted where?	<code>.Mounts</code>
What environment did it get?	<code>.Config.Env</code>

`.State.OOMKilled` is the one to reach for after an exit code of 137. It says plainly whether the kernel killed the process for memory, rather than leaving the number to be guessed at.

10. Files, stats and processes

These five answer questions that `logs` and `exec` cannot: what a container is consuming, which processes it is really running, and how to get a file in or out of it without rebuilding the image. They come up during an incident rather than during development, which is the wrong time to be reading the manual.

Command	What it does
<code>docker cp</code>	Copies files in or out of a container
<code>docker stats</code>	Live CPU, memory and network use
<code>docker top name</code>	The processes inside, as the host sees them
<code>docker port name</code>	The published port mapping
<code>docker diff name</code>	What changed in the writable layer

```
docker cp payroll:/app/logs/app.log ./app.log      # out of the container
docker cp ./application.yml payroll:/app/         # into the container
```

The container side always carries the `name:` prefix, and the side with the prefix is the container. That is the whole rule.

```
docker stats --no-stream
```

Output:

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O
a3f1c82b9e04	payroll	2.14%	612MiB / 2GiB	29.9%	1.2MB / 800kB
7d20e4b6c115	db	0.30%	118MiB / 2GiB	5.77%	800kB / 1.2MB

MEM USAGE / LIMIT is where a memory problem shows first. A JVM sitting at the limit and a rising restart count are the same story told twice.

11. Network commands

Command	What it does
<code>docker network ls</code>	Lists networks
<code>docker network create name</code>	Creates a user-defined bridge network
<code>docker network inspect name</code>	Shows which containers are attached
<code>docker network connect net name</code>	Attaches a running container
<code>docker network disconnect net name</code>	Detaches it
<code>docker network rm name</code>	Deletes it

```
docker network ls
```

Output:

NETWORK ID	NAME	DRIVER	SCOPE
b2c8f1a04e73	bridge	bridge	local
4f1a9c82b0de	host	host	local
9e0b3d71f5a2	none	null	local
c7d2e8a10b94	payroll_default	bridge	local

The first three come with Docker and cannot be removed. `payroll_default` is the one Compose made for its project.

Name resolution works on user-defined networks only. On the default `bridge` network, containers reach each other by address and not by name, which is the reason to create one.

12. Volume commands

Command	What it does
<code>docker volume ls</code>	Lists volumes
<code>docker volume create name</code>	Creates one
<code>docker volume inspect name</code>	Shows where it is on the host
<code>docker volume rm name</code>	Deletes it and its data
<code>docker volume prune</code>	Deletes every volume no container uses

A volume has no backup command. The way it is done is a throwaway container that mounts both the volume and a host directory:

```
docker run --rm \
  -v pgdata:/data \
  -v $(pwd):/backup \
  alpine tar czf /backup/pgdata.tar.gz -C /data .
```

`docker volume prune` **deletes data, permanently.** A volume that no container currently uses still holds the database from the stack that was stopped an hour ago.

13. Cleaning up

Docker never reclaims space on its own. It accumulates stopped containers, dangling images, unused volumes and the build cache until something is done.

```
docker system df
```

Output:

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	24	3	8.1GB	6.4GB (79%)
Containers	11	2	142MB	138MB (97%)
Local Volumes	7	2	3.2GB	2.1GB (65%)
Build Cache	186	0	4.7GB	4.7GB

Command	What it removes
<code>docker container prune</code>	Stopped containers
<code>docker image prune</code>	Dangling images, the <code><none></code> ones
<code>docker image prune -a</code>	Every image no container uses
<code>docker volume prune</code>	Volumes nothing is using
<code>docker builder prune</code>	The build cache
<code>docker system prune</code>	The first, second and fifth together
<code>docker system prune -a --volumes</code>	All of it

The `-a` changes the question. Without it, prune removes what is clearly rubbish. With it, prune removes anything not in use right now, which includes the base images every build depends on and the volume holding local test data.

14. Compose commands

Compose commands act on the whole project rather than one container.

Command	What it does
<code>docker compose up -d</code>	Creates and starts everything. <code>--build</code> rebuilds first
<code>docker compose down</code>	Removes the containers and the network. <code>-v</code> the volumes too
<code>docker compose ps</code>	The project's containers
<code>docker compose logs -f payroll</code>	One service's output
<code>docker compose exec payroll sh</code>	A shell in a running service
<code>docker compose restart payroll</code>	Restarts one service
<code>docker compose config</code>	Prints the file with variables resolved

`docker compose config` settles most Compose arguments. It shows the merged result of the base file, the override file and the `.env` values, which is the file Compose actually runs.

`exec` needs the service running; `run` does not. `docker compose run` starts a one-off container from a service definition, which is how a migration or a test suite is run.

```
docker compose run --rm payroll ./mvnw flyway:migrate
```

15. Commands that get confused

These two	Differ by
<code>docker stop</code> / <code>docker kill</code>	<code>stop</code> allows ten seconds, <code>kill</code> allows none
<code>docker rm</code> / <code>docker rmi</code>	Container against image
<code>docker run</code> / <code>docker start</code>	<code>run</code> makes a new container, <code>start</code> reuses one
<code>docker exec</code> / <code>docker attach</code>	A new process against the main one
<code>docker pull</code> / <code>docker build</code>	Fetches an image against making one
<code>docker ps</code> / <code>docker ps -a</code>	Running only against all
<code>docker prune</code> / <code>docker prune -a</code>	Unused rubbish against everything not running
<code>docker images</code> / <code>docker ps</code>	Images against containers

The one that costs real time is `run` against `start`. Running `docker run` a second time gives a second container, and the port is already taken by the first, so the error is about the port and the cause is the command.

```
docker run -d --name payroll -p 8080:8080 payroll:1.4 # the first time
docker start payroll                                # every time after
```

The pieces together

To do this	Use
Make an image	<code>docker build -t name:tag .</code>
Start a container	<code>docker run -d --name n -p 8080:8080 name:tag</code>
See what is running	<code>docker ps</code>
See what failed	<code>docker ps -a</code> , then <code>docker logs</code>
Look inside	<code>docker exec -it n sh</code>

To do this	Use
Restart it	<code>docker restart n</code>
Stop and delete it	<code>docker stop n</code> then <code>docker rm n</code>
Move an image	<code>docker pull</code> and <code>docker push</code>
Answer a detail	<code>docker inspect --format</code>
Get disk back	<code>docker system df</code> , then a prune
Run the whole stack	<code>docker compose up -d</code>

For example, a service that will not come up. `docker ps -a` shows it exited with 137. `docker inspect --format '{{.State.OOMKilled}}' payroll` says `true` , so it ran out of memory rather than crashing. `docker stats` on the rest of the stack shows the database at its limit too, and the fix is the memory limit in the Compose file rather than anything in the application.

The commands worth practicing are the four that answer questions: `ps -a` , `logs` , `exec` and `inspect` . The rest can be looked up, and `--help` is always there. Those four are what turn a container that "does not work" into a specific thing that went wrong.